SECURITY EAGLES

eJPTv2

Junior Penetration Tester Certification

STUDY GUIDE

www.securityeagles.com

FROM LEARNERS TO LEADERS

Vulnerability Assessment

eJPTv2 Certification Study Guide

From Learners to Leaders

Security Eagles

January 14, 2026

Contents

5 Vulnerability Assessment **2**
5.1 Understanding Vulnerabilities . . . 2
5.1.1 What is a Vulnerability? . . . 2
5.1.2 Vulnerability Classification . . . 2
5.1.3 CVSS Scoring System . . . 4
5.1.4 CVSS Calculator . . . 5
5.2 Vulnerability Databases . . . 6
5.2.1 Primary Databases . . . 6
5.2.2 Secondary Sources . . . 7
5.3 Using Searchsploit . . . 9
5.3.1 Basic Usage . . . 9
5.3.2 Essential Flags . . . 10
5.3.3 Advanced Techniques . . . 10
5.3.4 Database Maintenance . . . 11
5.4 Automated Vulnerability Scanning . . . 13
5.4.1 Scanning Methodologies . . . 13
5.4.2 Vulnerability Assessment Workflow . . . 13
5.4.3 Popular Scanning Tools . . . 13
5.4.4 Setup and Lab Environment . . . 14
5.4.5 Basic Scan Types . . . 15
5.4.6 Nessus Web Interface . . . 17
5.4.7 Analyzing Nessus Results . . . 18
5.4.8 Risk Matrix and Prioritization . . . 20
5.4.9 Scan Result Analysis . . . 25
5.4.10 Best Practices . . . 27
5.5 Chapter Summary . . . 29
5.5.1 Key Concepts Review . . . 29
5.5.2 Practical Exam Tips . . . 30
5.5.3 Common Pitfalls to Avoid . . . 32
5.5.4 Additional Resources . . . 32

5 Vulnerability Assessment

5.1 Understanding Vulnerabilities

5.1.1 What is a Vulnerability?

A **vulnerability** is a weakness in an information system, security procedures, internal controls, or implementation that could be exploited by a threat source.

Key Concepts:

- **Vulnerability:** The flaw itself (e.g., buffer overflow, SQL injection, misconfiguration)
- **Exploit:** The method or code used to leverage the vulnerability
- **Payload:** The malicious code executed after successful exploitation

5.1.2 Vulnerability Classification

Vulnerabilities can be classified by several criteria:

eJPTv2 Exam Focus

Key Areas for the Exam:

- Understanding CVSS scoring and severity levels
- Using searchsploit effectively during practical scenarios
- Interpreting vulnerability scanner output
- Identifying exploitable vulnerabilities
- Distinguishing between authenticated and unauthenticated scans
- Reading CVE identifiers and understanding their structure

Time Management Tip: During the practical exam, use `searchsploit -nmap` to quickly identify exploitable services from your reconnaissance phase.

By Severity

- **Critical:** Allows complete system compromise (e.g., unauthenticated RCE)
- **High:** Significant impact on confidentiality, integrity, or availability
- **Medium:** Moderate impact, may require specific conditions
- **Low:** Minimal impact, often requires local access or user interaction

By Type

- **Design Flaws:** Fundamental architectural weaknesses
  - Example: Insecure authentication mechanisms
  - Example: Lack of encryption in protocol design
- **Implementation Bugs:** Coding errors (buffer overflows, injection flaws)
  - Example: SQL Injection (SQLi) - CVE-2021-44228 (Log4Shell)
  - Example: Buffer Overflow - CVE-2014-0160 (Heartbleed)
- **Configuration Issues:** Improper system setup (default credentials, open ports)
  - Example: Default admin/admin credentials
  - Example: Exposed administrative interfaces
- **Missing Patches:** Unpatched known vulnerabilities
  - Example: Unpatched Windows SMB (MS17-010 EternalBlue)
  - Example: Outdated web server versions

Common Vulnerability Examples for eJPTv2

Frequently Encountered in Pentesting:

1. **vsFTPd 2.3.4 Backdoor** - CVE-2011-2523
   - Searchsploit ID: 17491
   - Exploit: Metasploit module available
   - Port: 21/tcp
2. **Apache Struts RCE** - CVE-2017-5638
   - CVSS: 10.0 (Critical)
   - Impact: Remote Code Execution
   - Common in Java web applications
3. **MS17-010 EternalBlue** - CVE-2017-0144
   - Target: Windows SMB protocol
   - Exploit: DoublePulsar + EternalBlue
   - Port: 445/tcp
4. **Shellshock** - CVE-2014-6271
   - Target: Bash shell
   - Vector: Environment variables
   - Common in CGI scripts

5.1.3 CVSS Scoring System

The Common Vulnerability Scoring System (CVSS) provides a standardized method for rating vulnerability severity.

CVSS v3.1 Base Metrics:

Metric	Description
Attack Vector (AV)	Network, Adjacent, Local, or Physical
Attack Complexity (AC)	Low or High
Privileges Required (PR)	None, Low, or High
User Interaction (UI)	None or Required
Scope (S)	Unchanged or Changed
Confidentiality (C)	None, Low, or High
Integrity (I)	None, Low, or High
Availability (A)	None, Low, or High

Score Ranges:

- **0.0:** None
- **0.1-3.9:** Low
- **4.0-6.9:** Medium
- **7.0-8.9:** High
- **9.0-10.0:** Critical

CVSS Example

CVE-2017-0144 (EternalBlue):
Vector: `CVSS:3.0/AV:N/AC:H/PR:N/UI:N/S:C/C:H/I:H/A:H`
Score: 8.1 (High)
Breakdown:

- Attack Vector: Network (remotely exploitable)
- Attack Complexity: High (requires specific conditions)
- Privileges Required: None (no authentication needed)
- User Interaction: None (fully automated)
- Scope: Changed (can affect other systems)
- Impact: High across all three (C/I/A)

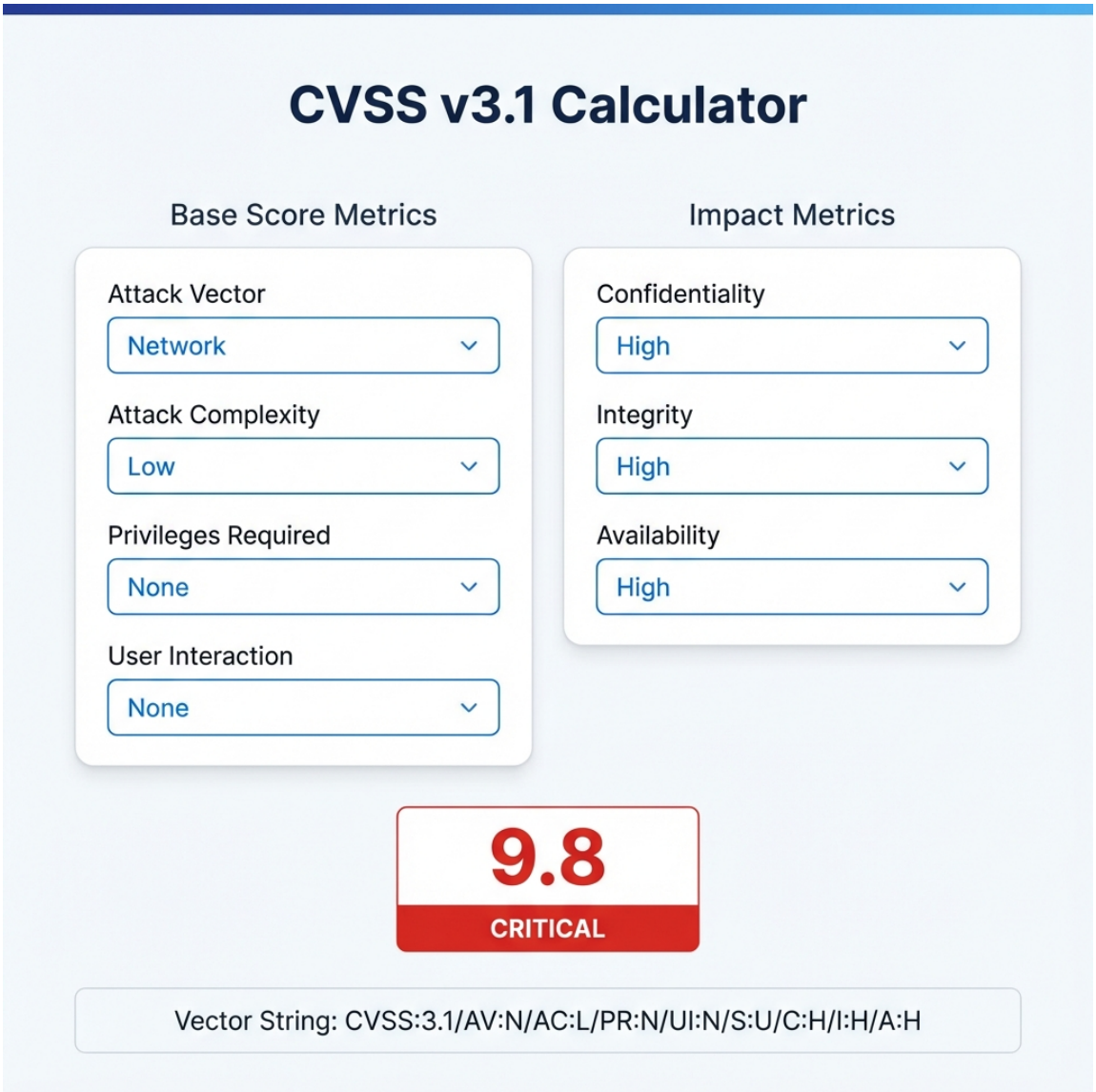

Figure 1: CVSS v3.1 Calculator Interface - Interactive tool for calculating vulnerability severity scores

5.1.4 CVSS Calculator

CVSS calculators help security professionals accurately score vulnerabilities. The following image shows a typical CVSS v3.1 calculator interface:

Using CVSS Calculators:

- **NVD Calculator:** https://nvd.nist.gov/vuln-metrics/cvss/v3-calculator
- **FIRST Calculator:** https://www.first.org/cvss/calculator/3.1
- Select each metric based on vulnerability characteristics
- Calculator generates vector string and numerical score
- Use for custom vulnerability assessments

5.2 Vulnerability Databases

5.2.1 Primary Databases

CVE (Common Vulnerabilities and Exposures)

The CVE system provides unique identifiers for publicly known vulnerabilities.

Format: `CVE-YEAR-NUMBER`

Example: `CVE-2021-44228` (Log4Shell)

What CVE Provides:

- Unique identifier
- Brief description
- References to advisories
- Publication date

NVD (National Vulnerability Database)

NIST's comprehensive vulnerability database that enhances CVE entries.

Additional Information:

- CVSS v2 and v3 scores
- CPE (Common Platform Enumeration) for affected products
- CWE (Common Weakness Enumeration) classification
- Patch information and references
- Exploit availability indicators

Access: `https://nvd.nist.gov/`

NVD Lookup Example

```
# Search NVD using curl (API)
curl "https://services.nvd.nist.gov/rest/json/cves/2.0?cveId=CVE
    -2021-44228"
```

Terminal Output - NVD API Response


```
{
  "cve": {
    "id": "CVE-2021-44228",
    "descriptions": [
      {
        "value": "Apache Log4j2 Remote Code Execution"
      }
    ],
    "metrics": {
      "cvssMetricV31": [
        {
          "cvssData": {
            "baseScore": 10.0,
            "baseSeverity": "CRITICAL"
          }
        }
      ]
    }
  }
}
```


Exploit-DB

The Exploit Database maintained by Offensive Security contains proof-of-concept exploits.

Contents:

- Verified and unverified exploit code
- Shellcode samples
- Security papers and documentation
- Google Hacking Database (GHDB)

Access: `https://www.exploit-db.com/`

5.2.2 Secondary Sources

CWE (Common Weakness Enumeration)

Categorizes the root causes of vulnerabilities:

- **CWE-79:** Cross-Site Scripting (XSS)
- **CWE-89:** SQL Injection
- **CWE-119:** Buffer Overflow
- **CWE-200:** Information Exposure

- **CWE-287:** Improper Authentication

Other Resources

- **Packet Storm:** Security tools and exploits
- **SecurityFocus:** Vulnerability database and mailing lists
- **VulnDB:** Commercial vulnerability intelligence
- **GitHub Security Advisories:** Open-source project vulnerabilities

Cross-Referencing

Always cross-reference vulnerabilities across multiple databases:

1. Scanner identifies vulnerability
2. Check NVD for CVSS score and details
3. Search Exploit-DB for available exploits
4. Review vendor advisories for patches
5. Check CWE for vulnerability class

5.3 Using Searchsploit

`searchsploit` is the command-line interface for the Exploit Database, providing offline access to thousands of exploits.

5.3.1 Basic Usage

Installation:

```
# Included with Kali Linux
# Or install separately:
sudo apt install exploitdb
```

Basic Search:

```
searchsploit <search term>
```

Searchsploit Terminal Example

```
root@kali:~# searchsploit apache 2.4

-----------------------------------------
    ---------------------------------
 Exploit Title                          |  Path
-----------------------------------------
    ---------------------------------
Apache 2.4.x - 'mod_ssl' Remote Denial  | linux/dos/47688.sh
Apache 2.4.7 - 'mod_status' Scoreboard  | linux/dos/34133.txt
Apache 2.4.17 < 2.4.38 - 'apache_mod_s  | linux/local/46676.c
Apache 2.4.49 - Path Traversal & Remote | multiple/webapps/50383.sh
Apache 2.4.50 - Remote Code Execution   | multiple/webapps/50512.py
Apache HTTP Server 2.4.49 - Path Trave  | multiple/webapps/50406.sh
-----------------------------------------
    ---------------------------------
Shellcodes: No Results
```

Terminal Output - Searchsploit Output

```
root@kali:~# searchsploit apache 2.4

--------------------------------------- ---------------------------------
 Exploit Title                          |  Path
--------------------------------------- ---------------------------------
Apache 2.4.x - 'mod_ssl' Remote Denial  | linux/dos/47688.sh
Apache 2.4.7 - 'mod_status' Scoreboard  | linux/dos/34133.txt
Apache 2.4.17 < 2.4.38 - 'apache_mod_s  | linux/local/46676.c
Apache 2.4.49 - Path Traversal & Remote | multiple/webapps/50383.sh
Apache 2.4.50 - Remote Code Execution   | multiple/webapps/50512.py
Apache HTTP Server 2.4.49 - Path Trave  | multiple/webapps/50406.sh
--------------------------------------- ---------------------------------
Shellcodes: No Results
```

5.3.2 Essential Flags

Flag	Function	Description
`-t`	Title search	Search only in exploit titles
`-e`	Exact match	Require exact phrase match
`-m ID`	Mirror	Copy exploit to current directory
`-x ID`	Examine	View exploit code in pager
`-p ID`	Path	Show full filesystem path
`-w ID`	Web	Display Exploit-DB URL
`-c`	Case sensitive	Enable case-sensitive search
`-nmap file`	Nmap integration	Search from Nmap XML output
`-u`	Update	Update local database

5.3.3 Advanced Techniques

Filtering Results

Precision Searching

```
# Exclude Denial of Service exploits
searchsploit Apache | grep -v "DoS"

# Find only remote exploits
searchsploit Windows SMB | grep "Remote"

# Exact version match
searchsploit -e "ProFTPd 1.3.5"

# Case-sensitive search
searchsploit -c "WordPress"
```

Working with Exploits

Exploit Workflow

```
# 1. Search for exploit
searchsploit vsftpd 2.3.4

# 2. Copy exploit to current directory
searchsploit -m 17491

# 3. Examine the exploit code
cat 17491.rb

# 4. View online reference
searchsploit -w 17491

# 5. Get full path
searchsploit -p 17491
```

Nmap Integration

Automated Vulnerability Mapping

```
# Step 1: Perform version detection scan
nmap -sV -oX scan_results.xml 192.168.1.0/24

# Step 2: Auto-search all discovered services
searchsploit --nmap scan_results.xml
```

This automatically searches for exploits matching all service versions found in the Nmap scan.

5.3.4 Database Maintenance

Keeping Searchsploit Updated

Update the local database before every engagement:

```
# Update database
sudo searchsploit -u

# Verify update
searchsploit --version

# Check database location
ls -lh /usr/share/exploitdb/
```

Update Frequency: Weekly for active pentesters, monthly minimum for others

Terminal Output - Searchsploit Update

```
root@kali:~# searchsploit -u

[i] Updating Exploit-DB from GitHub...
[i] Cloning latest exploitdb...
[i] Updating exploitdb...
[i] Exploit Database updated successfully!
```

Exploit Safety

Never run exploits without understanding them!
Before using any exploit:

1. Read the entire source code
2. Understand what it does
3. Check for hardcoded IP addresses or payloads
4. Verify it targets the correct version
5. Test in isolated environment first
6. Modify as needed for your target

5.4 Automated Vulnerability Scanning

5.4.1 Scanning Methodologies

Authenticated vs. Unauthenticated Scanning

Aspect	Unauthenticated	Authenticated
Perspective	External attacker view	Internal/privileged view
Credentials	None provided	Valid credentials supplied
Detection Depth	Network services only	Full system access
Patch Verification	Version banner-based	Direct patch check
False Positives	High (30-40%)	Low (5-10%)
Use Case	External pentest	Internal audit, compliance

5.4.2 Vulnerability Assessment Workflow

A systematic approach to vulnerability assessment ensures comprehensive coverage and accurate results:

Workflow Stages:

1. **Asset Discovery:** Identify all systems and services
2. **Vulnerability Scanning:** Automated and manual testing
3. **Analysis & Validation:** Verify and prioritize findings
4. **Risk Assessment:** Evaluate business impact
5. **Reporting:** Document vulnerabilities and recommendations
6. **Remediation:** Apply patches and fixes
7. **Verification:** Confirm vulnerabilities are resolved
8. **Continuous Monitoring:** Ongoing assessment cycle

5.4.3 Popular Scanning Tools

Nessus

Commercial vulnerability scanner from Tenable.

Key Features:

- Comprehensive vulnerability database
- Authenticated and unauthenticated scanning
- Compliance auditing (PCI-DSS, HIPAA, CIS)

Figure 2: Complete Vulnerability Assessment Workflow - From discovery to remediation tracking

- Custom policy creation
- Detailed reporting

5.4.4 Setup and Lab Environment

Installing Nessus

For detailed download, installation, and initial setup instructions, refer to the official Tenable Nessus documentation:
`https://docs.tenable.com/nessus/Content/GettingStarted.htm`

Setting up a Vulnerable Target (DVWA)

To safely practice scanning, use the Damn Vulnerable Web App (DVWA) running in a Docker container.

Running DVWA with Docker

```
# 1. Pull the DVWA image
sudo docker pull vulnerables/web-dvwa

# 2. Run the container
# Maps host port 80 to container port 80
sudo docker run --rm -it -p 80:80 vulnerables/web-dvwa
```

Access: Open `http://localhost` or `http://127.0.0.1` in your browser.
Default Credentials:

- **Username:** admin
- **Password:** password

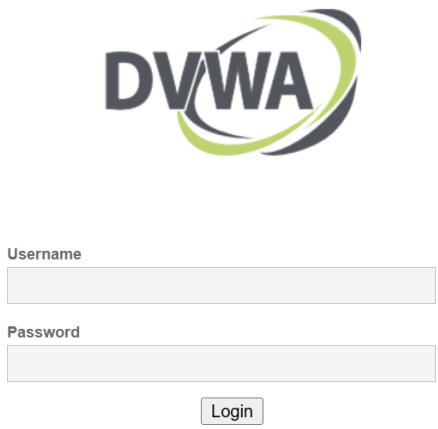

Figure 3: Running DVWA Container - Terminal output showing the successful startup of the DVWA service

5.4.5 Basic Scan Types

- Basic Network Scan
- Credentialed Patch Audit
- Web Application Tests

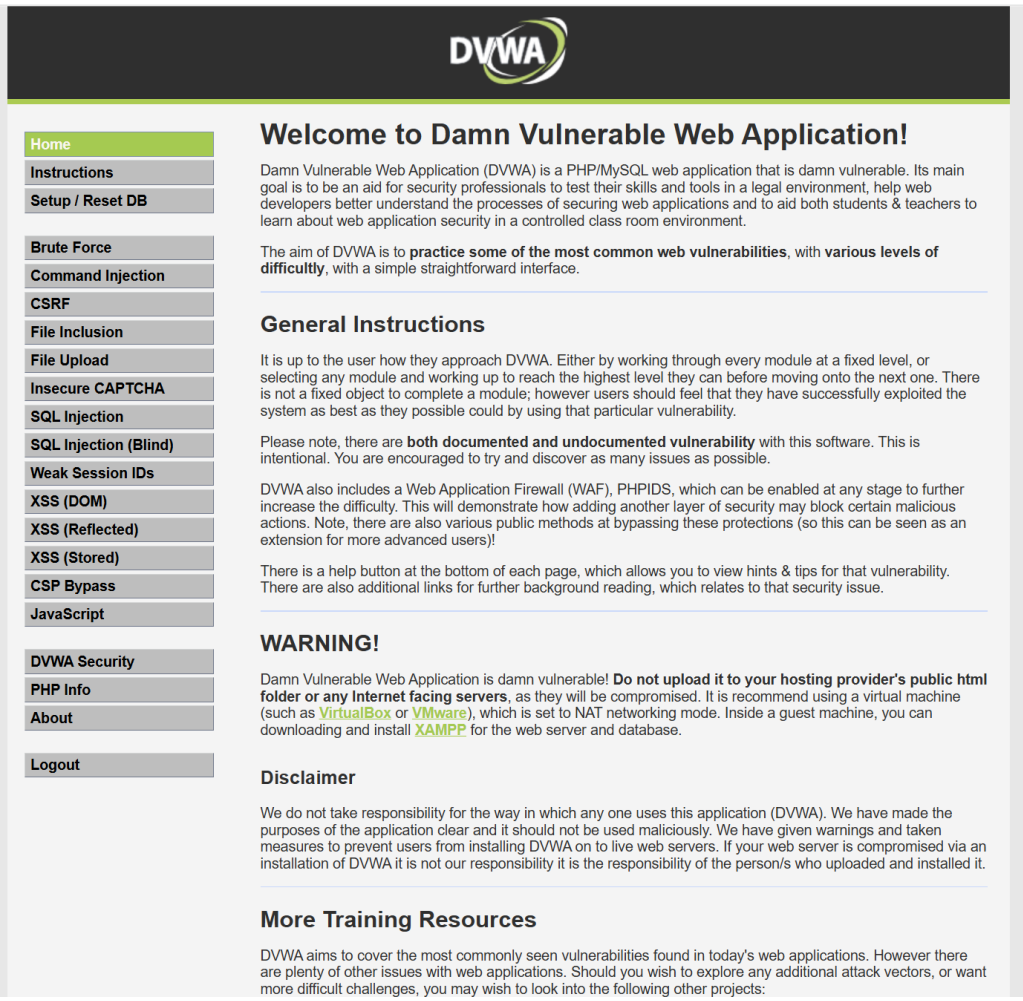

Figure 4: DVWA Login Page - The web interface accessible at localhost:80 after starting the container

- Malware Scan
- Policy Compliance

Choosing the Right Scan Template:

- **Host Discovery:** Quick network sweep to identify live hosts
- **Basic Network Scan:** General-purpose vulnerability scan for all hosts
- **Web Application Tests:** Specialized web vulnerability scanning
- **Credential Validation:** Verify provided credentials work correctly
- **Advanced Scan:** Custom configuration for specific requirements
- **Credentialed Patch Audit:** Authenticated scan for patch verification
- **Malware Scan:** Detect malware on Windows and Unix systems

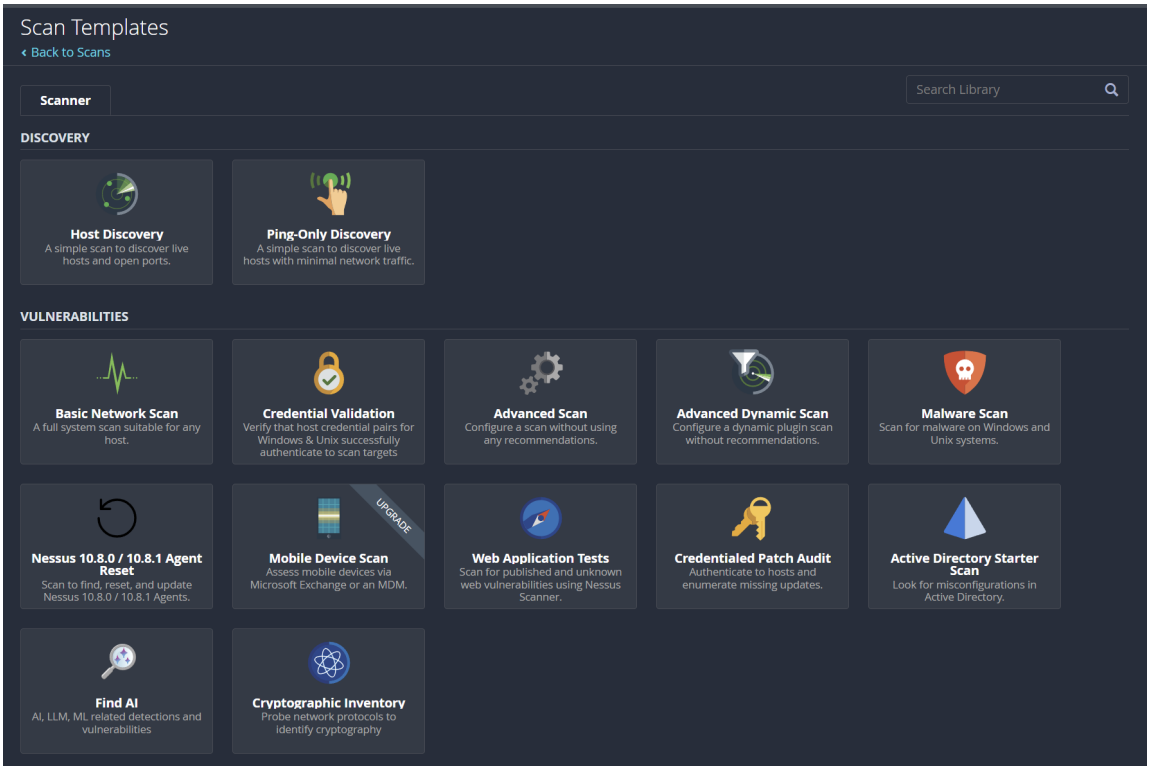

Figure 5: Nessus Scan Templates - Various scan types available including Host Discovery, Basic Network Scan, Web Application Tests, Credential Validation, and more

Nessus CLI Scan

```
# List available policies
/opt/nessus/bin/nessuscli policy list

# Start a scan
/opt/nessus/bin/nessuscli scan new \
  --name="Network Scan" \
  --targets="192.168.1.0/24" \
  --policy="Basic Network Scan"
```

5.4.6 Nessus Web Interface

Nessus provides a modern web-based interface for managing scans. The following screenshot shows a typical scan overview:

Key Interface Elements:

- **Scan Status:** Real-time progress indicator (Running/Complete)
- **Policy Type:** Shows the scan template used (Web Application Tests)
- **Severity Base:** CVSS scoring version (v3.0)
- **Vulnerability Bar:** Color-coded severity distribution

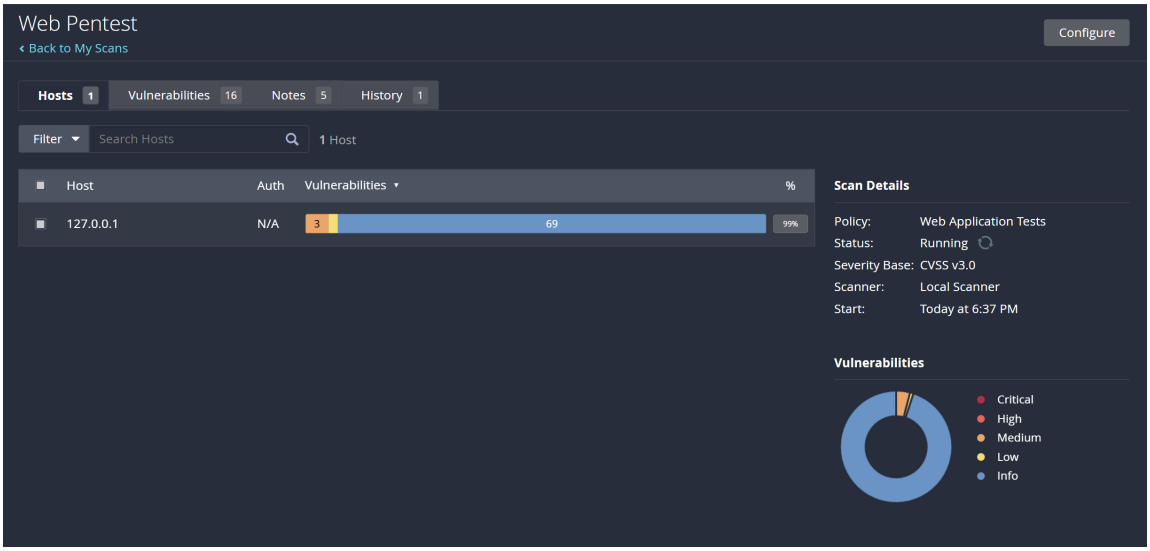

Figure 6: Nessus Web Pentest Scan Overview - Shows scan status, policy configuration, and vulnerability distribution with 3 Medium and 69 Info findings

- **Percentage Complete:** Scan progress tracking (99%)
- **Host Count:** Number of targets scanned

5.4.7 Analyzing Nessus Results

Once a scan completes, Nessus provides detailed vulnerability information organized by severity:

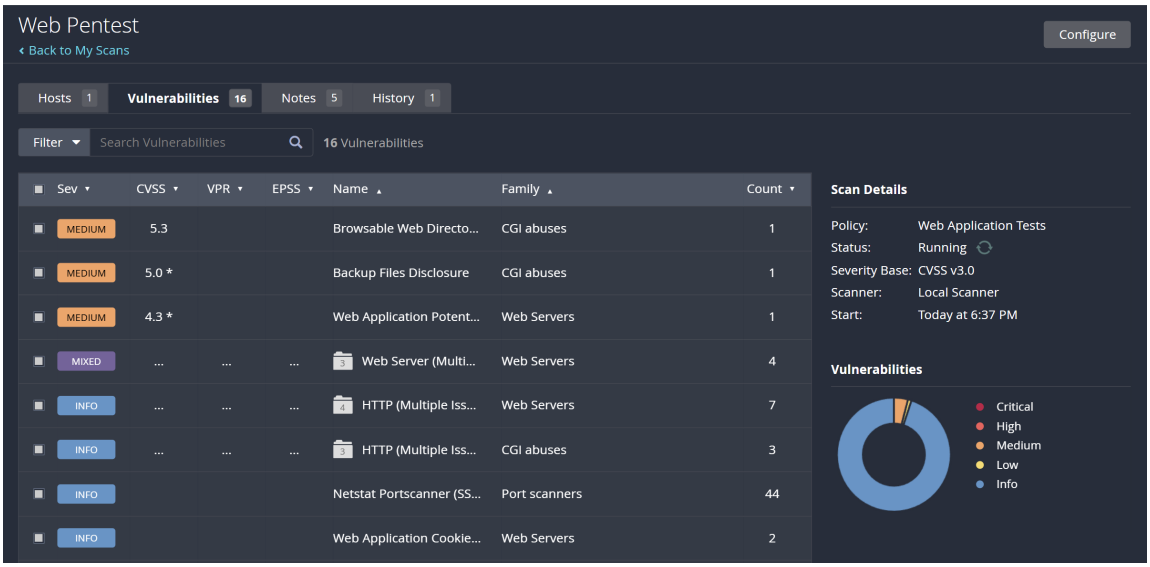

Figure 7: Nessus Vulnerabilities List - Detailed view showing CVSS scores, vulnerability names, families, and occurrence counts

Understanding the Vulnerabilities Tab:

- **Sev (Severity):** Color-coded severity levels (Critical, High, Medium, Low, Info)

- **CVSS:** Numerical score based on CVSS v3.0 standard
- **VPR:** Vulnerability Priority Rating (Tenable's risk-based scoring)
- **EPSS:** Exploit Prediction Scoring System percentage
- **Name:** Vulnerability title (e.g., "Browsable Web Directories")
- **Family:** Vulnerability category (CGI abuses, Web Servers, Port scanners)
- **Count:** Number of affected instances across scanned hosts

Common Findings in Example:

- **Browsable Web Directories:** CVSS 5.3 - Allows directory listing
- **Backup Files Disclosure:** CVSS 5.0 - Exposed backup files
- **Web Application Potential Vulnerabilities:** CVSS 4.3 - Various web issues
- **HTTP Multiple Issues:** Informational findings for web servers
- **Netstat Portscanner:** 44 instances - Network service enumeration

Terminal Output - Nessus Scan Results

```
root@kali:~# nessuscli report export --format=text scan_001

========================================
Nessus Vulnerability Scan Report
========================================
Scan Name: Network Scan
Targets: 192.168.1.0/24
Completed: 2024-01-08 20:00:00

Summary:
  Critical: 3
  High: 12
  Medium: 45
  Low: 89
  Info: 234

Top Vulnerabilities:
  [CRITICAL] MS17-010: SMBv1 Remote Code Execution
    Hosts: 192.168.1.10, 192.168.1.15
    CVSS: 10.0

  [CRITICAL] Apache Struts RCE (CVE-2017-5638)
    Hosts: 192.168.1.100
    CVSS: 10.0

  [HIGH] SSL/TLS POODLE Vulnerability
    Hosts: 192.168.1.50, 192.168.1.51
    CVSS: 7.5
```

Nessus Reporting Features

Nessus provides comprehensive reporting capabilities:

- **Executive Summary:** High-level overview for management
- **Technical Report:** Detailed findings for security teams
- **Compliance Reports:** PCI-DSS, HIPAA, ISO 27001 audits
- **Custom Templates:** Tailored reports for specific needs
- **Export Formats:** PDF, HTML, CSV, XML
- **Trend Analysis:** Historical vulnerability data

OpenVAS

Open-source vulnerability scanner (Greenbone).

5.4.8 Risk Matrix and Prioritization

Vulnerability prioritization requires considering both likelihood and impact. The risk matrix helps visualize and prioritize remediation efforts:

Risk Prioritization Framework:

Risk Level	Characteristics	SLA	Action
Critical	High likelihood + High impact	24-48 hours	Immediate patching
High	Medium-High likelihood/impact	1-2 weeks	Priority remediation
Medium	Medium likelihood + Medium impact	1-3 months	Scheduled patching
Low	Low likelihood or Low impact	3-6 months	Review and plan
Informational	No direct risk	Monitor	Document only

Risk Calculation Factors:

- **CVSS Score:** Technical severity (0-10)
- **Exploitability:** Public exploit availability
- **Asset Criticality:** Business importance of affected system
- **Exposure:** Internet-facing vs internal
- **Compensating Controls:** Existing security measures
- **Threat Intelligence:** Active exploitation in the wild

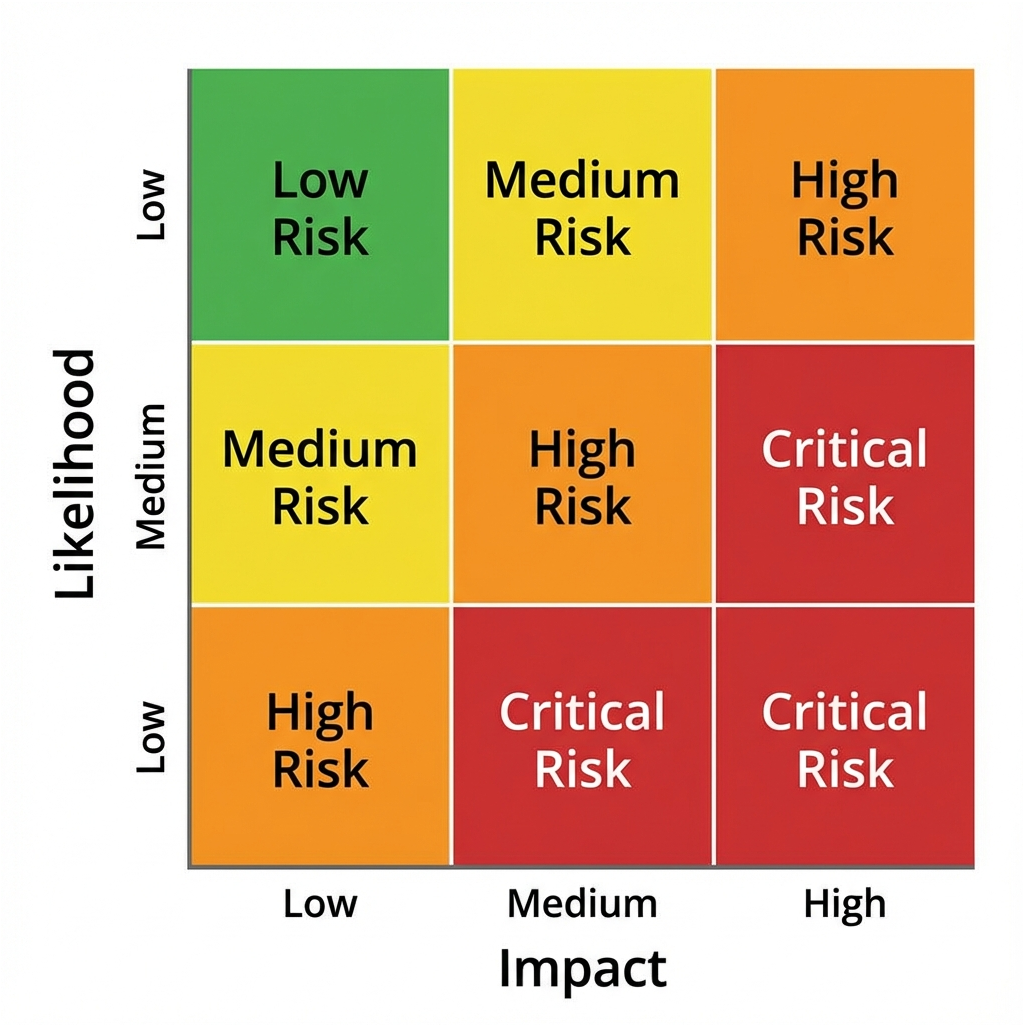

Figure 8: Risk Assessment Matrix - Prioritizing vulnerabilities based on likelihood and impact

OpenVAS

Open-source vulnerability scanner (Greenbone).

Key Features:

- Free and open-source
- Regular NVT (Network Vulnerability Tests) updates
- Web-based interface
- Comprehensive scanning capabilities
- Scheduled scanning

OpenVAS CLI Scan

```
# Create a new target
omp --username admin --password admin \
  --xml="<create_target>
    <name>Internal Network</name>
    <hosts>192.168.1.0/24</hosts>
  </create_target>"

# Start scan task
omp --username admin --password admin \
  --xml="<create_task>
    <name>Full Scan</name>
    <target id='target-uuid'/>
    <config id='full-and-fast'/>
  </create_task>"
```

Terminal Output - OpenVAS Scan Summary

```
root@kali:~# omp --get-results task-uuid

========================================
OpenVAS Scan Results
========================================
Task: Full Scan
Status: Done
Progress: 100%

Vulnerability Summary:
  High: 8
  Medium: 23
  Low: 67

Top Findings:
  [HIGH] Heartbleed OpenSSL Vulnerability
    Host: 192.168.1.100
    Port: 443/tcp
    CVE: CVE-2014-0160

  [HIGH] Shellshock Bash Vulnerability
    Host: 192.168.1.50
    Port: 80/tcp
    CVE: CVE-2014-6271

  [MEDIUM] Outdated Apache Version
    Host: 192.168.1.100
    Port: 80/tcp
    Version: 2.4.6
```

Nikto

Web server vulnerability scanner.

Nikto Usage

```
# Basic web server scan
nikto -h http://target.com

# Scan with specific port
nikto -h target.com -p 8080

# Save output to file
nikto -h target.com -o scan_results.html -Format html

# Scan multiple hosts
nikto -h hosts.txt
```

Terminal Output - Nikto Scan Results

```
root@kali:~# nikto -h http://192.168.1.100

- Nikto v2.1.6
---------------------------------------------------------------------------
+ Target IP:          192.168.1.100
+ Target Hostname:    webserver.local
+ Target Port:        80
+ Start Time:         2024-01-08 20:30:00
---------------------------------------------------------------------------
+ Server: Apache/2.4.41 (Ubuntu)
+ The anti-clickjacking X-Frame-Options header is not present.
+ The X-XSS-Protection header is not defined.
+ Apache/2.4.41 appears to be outdated (current is 2.4.54)
+ OSVDB-3268: /admin/: Directory indexing found.
+ OSVDB-3092: /admin/: This might be interesting...
+ OSVDB-3233: /icons/README: Apache default file found.
+ 7535 requests: 0 error(s) and 6 item(s) reported
+ End Time:           2024-01-08 20:45:00 (15 minutes)
```

Nmap NSE Scripts

Nmap Scripting Engine for vulnerability detection.

Nmap Vulnerability Scanning

```
# Run all vulnerability scripts
nmap --script vuln target.com

# Specific vulnerability check
nmap --script smb-vuln-ms17-010 192.168.1.0/24

# HTTP vulnerability scan
nmap --script http-vuln-* target.com

# SSL/TLS vulnerability check
nmap --script ssl-* -p 443 target.com
```

Terminal Output - Nmap NSE Vulnerability Scan

```
root@kali:~# nmap --script vuln 192.168.1.100

Starting Nmap 7.92 ( https://nmap.org )
Nmap scan report for 192.168.1.100
Host is up (0.0012s latency).

PORT     STATE SERVICE
21/tcp   open  ftp
| ftp-vsftpd-backdoor:
|   VULNERABLE:
|   vsFTPd version 2.3.4 backdoor
|     State: VULNERABLE (Exploitable)
|     IDs:  CVE:CVE-2011-2523
|     Risk factor: High

22/tcp   open  ssh

80/tcp   open  http
| http-csrf:
| Spidering limited to: maxdepth=3
|   Found the following possible CSRF vulnerabilities:
|     Path: http://192.168.1.100/admin/login.php
|_    Form id: login-form
| http-enum:
|   /admin/: Possible admin folder
|   /backup/: Backup folder
|_  /phpmyadmin/: phpMyAdmin

445/tcp open  microsoft-ds
| smb-vuln-ms17-010:
|   VULNERABLE:
|   Remote Code Execution vulnerability in Microsoft SMBv1
|     State: VULNERABLE
|     IDs:  CVE:CVE-2017-0143
|     Risk factor: HIGH
|       A critical remote code execution vulnerability exists
|       in Microsoft SMBv1 servers (ms17-010).
|
|     Disclosure date: 2017-03-14
|     References:
|       https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2017-0143
|_      https://technet.microsoft.com/en-us/library/security/ms17-010

Nmap done: 1 IP address (1 host up) scanned in 45.23 seconds
```

5.4.9 Scan Result Analysis

Triage Workflow

1. **Filter by Severity:** Focus on Critical and High findings first
2. **Verify Exploitability:** Check if public exploits exist

3. **Assess Business Impact:** Consider asset criticality
4. **Manual Verification:** Confirm findings aren't false positives
5. **Document Evidence:** Screenshot, logs, proof-of-concept
6. **Prioritize Remediation:** Create action plan with timelines

False Positive Identification

Common Causes:

- Backported patches (version number unchanged)
- Banner-based detection without actual testing
- Misconfigured scanner settings
- Outdated vulnerability signatures

Verification Methods:

- Manual testing with safe payloads
- Checking patch levels directly
- Reviewing vendor security bulletins
- Consulting system administrators

Manual Verification

Scenario: Scanner reports "Apache 2.4.6 vulnerable to CVE-2017-7679"
Verification Steps:

```
# Check actual Apache version
curl -I http://target.com | grep Server

# Check if patch is backported (RHEL/CentOS)
rpm -q --changelog httpd | grep CVE-2017-7679

# Manual test (if safe)
curl -H "Range: bytes=0-" http://target.com/test.txt
```

Result: If patch is backported, mark as False Positive

Terminal Output - Manual Verification Output

```
root@kali:~# curl -I http://target.com | grep Server

Server: Apache/2.4.6 (CentOS)

root@kali:~# rpm -q --changelog httpd | grep CVE-2017-7679

* Wed Jun 14 2017 - Security fix for CVE-2017-7679
  - Backported patch from upstream

root@kali:~# echo "Patch is backported - FALSE POSITIVE"

Patch is backported - FALSE POSITIVE
```

5.4.10 Best Practices

Scanning Best Practices

- **Schedule Appropriately:** Scan during maintenance windows
- **Use Credentials:** Authenticated scans are more accurate
- **Update Regularly:** Keep vulnerability databases current
- **Tune Scanners:** Disable checks that cause issues
- **Verify Findings:** Always manually confirm critical findings
- **Document Everything:** Maintain scan history and trends
- **Coordinate:** Inform IT teams before scanning

Scanning Risks

Vulnerability scanning can cause:

- Service disruptions or crashes
- Network congestion
- IDS/IPS alerts
- Account lockouts
- Data corruption (in rare cases)

Always:

- Obtain written authorization
- Test in non-production first
- Have rollback plans
- Monitor during scans

5.5 Chapter Summary

5.5.1 Key Concepts Review

1. Vulnerability Fundamentals:

- Vulnerability = Weakness in system or implementation
- Exploit = Method to leverage the vulnerability
- Payload = Malicious code executed after exploitation
- Risk = Vulnerability × Threat × Asset Value

2. CVSS Scoring:

- Score Range: 0.0 (None) to 10.0 (Critical)
- Base Metrics: AV, AC, PR, UI, S, C, I, A
- Use CVSS calculators for accurate scoring
- Consider temporal and environmental metrics for complete assessment

3. Essential Databases:

- **CVE:** Unique vulnerability identifiers
- **NVD:** Comprehensive vulnerability data with CVSS scores
- **Exploit-DB:** Proof-of-concept exploits and shellcode
- **CWE:** Vulnerability classification by weakness type

4. Searchsploit Mastery:

- Essential for offline exploit searching
- Use `-m` to copy exploits, `-x` to examine code
- `-nmap` integration automates vulnerability mapping
- Always update before engagements: `searchsploit -u`

5. Automated Scanning:

- **Authenticated Scans:** More accurate, fewer false positives
- **Unauthenticated Scans:** External attacker perspective
- **Tools:** Nessus (commercial), OpenVAS (open-source), Nikto (web)
- **Nmap NSE:** Targeted vulnerability checks with scripts

5.5.2 Practical Exam Tips

eJPTv2 Exam Workflow

Recommended Vulnerability Assessment Workflow:
Phase 1: Discovery (15-20 minutes)

```
# 1. Network sweep
nmap -sn 10.10.10.0/24 -oA hosts

# 2. Service enumeration
nmap -sV -sC -p- -oX services.xml 10.10.10.0/24

# 3. Quick vulnerability check
nmap --script vuln -oN vulns.txt <target_IPs>
```

Phase 2: Vulnerability Identification (20-30 minutes)

```
# 4. Search for exploits
searchsploit --nmap services.xml

# 5. Specific service checks
searchsploit <service_name> <version>

# 6. Copy promising exploits
searchsploit -m <exploit_ID>
```

Phase 3: Exploitation (30-40 minutes)

```
# 7. Review and modify exploit
cat <exploit_file>

# 8. Test exploitation
# (Follow exploit instructions)

# 9. Document findings
screenshot + commands + output
```

Quick Reference Commands

Must-Know Commands for eJPTv2:

```
# Update exploit database
sudo searchsploit -u

# Search for exploits
searchsploit <keyword>

# Copy exploit to current directory
searchsploit -m <ID>

# Get Exploit-DB URL
searchsploit -w <ID>

# Nmap integration
searchsploit --nmap <scan.xml>

# Vulnerability scan with Nmap
nmap --script vuln <target>

# Specific vulnerability check
nmap --script <vuln-name> <target>

# Web server scan
nikto -h http://<target>
```

5.5.3 Common Pitfalls to Avoid

Exam Mistakes

Don't Make These Mistakes:

1. **Skipping Version Detection:**
   - Always use `nmap -sV` for accurate service versions
   - Version information is critical for exploit selection
2. **Not Reading Exploit Code:**
   - Read the exploit before running it
   - Check for required modifications (IP, port, payload)
   - Understand what the exploit does
3. **Ignoring False Positives:**
   - Verify scanner findings manually
   - Test vulnerabilities before reporting
   - Check for backported patches
4. **Poor Time Management:**
   - Don't spend too long on one vulnerability
   - Prioritize high-severity, easily exploitable findings
   - Save documentation for the end
5. **Outdated Exploit Database:**
   - Update searchsploit before exam
   - Keep Metasploit framework updated
   - Refresh Nmap NSE scripts

5.5.4 Additional Resources

Online Resources:

- **NVD:** https://nvd.nist.gov/
- **Exploit-DB:** https://www.exploit-db.com/
- **CVE Details:** https://www.cvedetails.com/
- **CVSS Calculator:** https://www.first.org/cvss/calculator/3.1
- **Packet Storm:** https://packetstormsecurity.com/
- **Security Focus:** https://www.securityfocus.com/

Practice Labs:

- **HackTheBox:** Realistic vulnerable machines
- **TryHackMe:** Guided vulnerability exercises
- **VulnHub:** Downloadable vulnerable VMs
- **Metasploitable 2/3:** Intentionally vulnerable Linux VMs
- **DVWA:** Damn Vulnerable Web Application

Books and References:

- *The Web Application Hacker's Handbook*
- *Metasploit: The Penetration Tester's Guide*
- *OWASP Testing Guide*
- *Penetration Testing: A Hands-On Introduction to Hacking*

Good luck with your eJPTv2 certification!

Remember: Practice makes perfect. The more you scan, the better you become at identifying and exploiting vulnerabilities.

STAY TUNED FOR MORE

www.securityeagles.com

SECURITY EAGLES

FROM LEARNERS TO LEADERS